USEI09 Proximity Search (Nearest-N)

Context

Operations need to find the N closest stations to a target coordinate (e.g., for finding the nearest major hubs). A 2D KD-tree (k=2) on (latitude, longitude) supports nearest-neighbor operations much faster than scanning all stations sequentially.

User Story

As an analyst, I want to find the nearest N stations to a given (lat, lon) using the 2D-tree, with optional filters, so I can get accurate nearby results efficiently.

Acceptance Criteria

- Use Haversine distance (km) with Earth radius between two (lat, lon) points.
- The search should enable optional filtering based on the time zone criteria.
- **Returns:** A list with N nearest neighbours; Temporal analysis complexity.

Temporal Complexity Analysis

1. k-NN Algorithm Logic

The search is performed recursively using a Branch and Bound approach with a Max-Heap (Priority Queue) of size N:

1. **Descend:** Traverse the tree to the leaf node closest to the target.
2. **Update:** Calculate distance. If the current node is closer than the farthest node in the Heap (or if Heap < N), add it.
3. **Prune:** As the recursion unwinds, check if the "bad" side (the subtree on the other side of the splitting plane) could theoretically contain a closer point. If the distance to the splitting plane is greater than the worst distance in the Heap, the entire subtree is discarded (pruned).

Detailed Complexity Analysis (Code Excerpt)

```
private void findNearestNRecursive(Node node, double target, int N,
PriorityQueue maxHeap) {
    if (node == null) return;

    // 1. Update Heap (Cost: O(log N))
    double dist = haversineDistance(target, node.coords);
    if (dist < maxHeap.peek().distance()) {
        maxHeap.poll();
        maxHeap.add(new Result(dist, node));
    }
    // 2. Determine Search Order
    Node goodSide = (diff < 0) ? node.left : node.right;
    Node badSide  = (diff < 0) ? node.right : node.left;
    // 3. Recurse Good Side (Cost: O(log n) depth)
    findNearestNRecursive(goodSide, ...);
    // 4. Pruning Check
    // If the hypersphere of the N-th neighbor intersects the splitting plane,
    // we must search the bad side.
    if (distToSplittingPlane < maxHeap.peek().distance()) {
        findNearestNRecursive(badSide, ...);
    }
}
```

Complexity Scenarios

Best / Average Case: If N is small relative to n, and the optional Filter matches stations in the geometric vicinity of the target.

Justification: The algorithm quickly locates valid neighbors. The distance to the N-th neighbor becomes a tight pruning threshold. As this threshold shrinks, the algorithm discards large sections of the tree because their bounding boxes are farther away than the neighbors already found.

Worst Case: Occurs with strict filtering such asTimeZone mismatch or a large N, forcing a full tree traversal with priority queue updates.

1. **Strict Filtering:** Searching for "Asia/Tokyo" stations while targeting coordinates in Portugal. The algorithm visits local nodes but rejects them. It cannot shrink the pruning threshold and traverses the entire tree.
2. **Large N:** Requesting N approx n.

- **Justification:** The pruning check (`distToSplittingPlane < maxDist`) almost always evaluates to true, forcing the recursion to visit both Left and Right children at every node. The log N factor comes from the Priority Queue operations at every visited node.